



# Tema 1. Introducción a la informática y conceptos básicos

**Grupo A:**  
Fernando Rojas  
frojas@ugr.es

**Grupo B:**  
Samuel Romero  
sromero@ugr.es

**Grupo C:**  
Juan Luis Jiménez  
juanlu@ugr.es

# Tema 1 · Nuestro recorrido

- 01**    **Definiciones básicas**  
Información, bits y códigos
- 02**    **Unidades funcionales**  
El computador por dentro
- 03**    **Tipos de computadores**  
Distintos equipos y usos
- 04**    **Software de un computador**  
Programas que lo hacen funcionar

# L1.1

---

## Definiciones básicas

01

# ¿Qué es la informática?

El estudio del tratamiento automático de la información mediante computadores.

**REPRESENTAR**

**ORGANIZAR**

**PROCESAR**

La informática estudia cómo representar, organizar y procesar información de forma automática.

1 / 19

La informática estudia el tratamiento automático de la información mediante computadores. Representar consiste en codificar la información; organizar, en estructurarla y almacenarla; procesar, en aplicar operaciones para obtener resultados. La siguiente diapositiva introduce el computador como máquina que permite realizar estas tareas. Se mantiene una explicación general, sin el ejemplo de una compra.

## ¿Qué es un computador?

Una máquina electrónica programable que trata información.



**RECIBE** datos

**PROCESA** con instrucciones

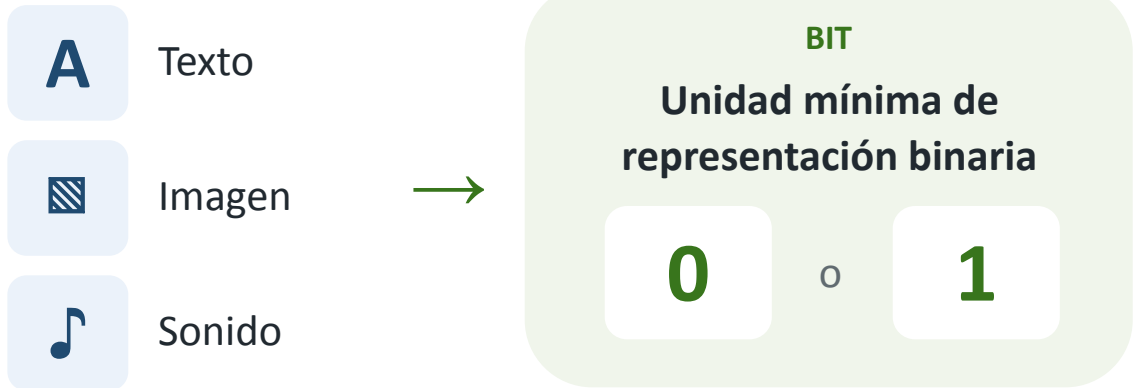
**ALMACENA** datos y resultados

**PROPORCIONA** resultados

Informática: tratamiento automático de la información. Un computador es una máquina electrónica programable que recibe, procesa, almacena y proporciona datos. Ejemplo oral: recibe unidades y precio, calcula el importe, lo guarda y lo muestra. La ilustración anticipa componentes, sin explicarlos todavía.

## Información digital, representación binaria

Texto, imágenes y sonidos se representan mediante secuencias de bits.



3 / 19

Un bit es la unidad mínima de representación binaria: admite 0 o 1. La información completa se representa con secuencias de bits. Los computadores actuales son digitales y binarios. Digital significa valores discretos; no equivale por definición a binario.

## Combinaciones con 1, 2 y 3 bits

Cada bit adicional duplica el número de combinaciones.

### 1 BIT

0

1

2 combinaciones

### 2 BITS

00

01

10

11

4 combinaciones

### 3 BITS

000

001

010

011

100

101

110

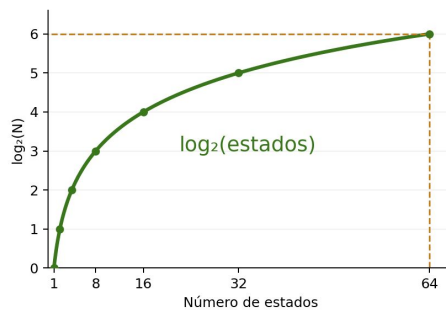
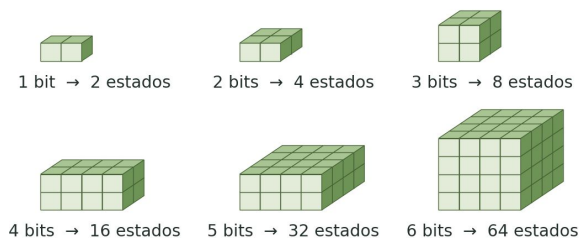
111

8 combinaciones

Se enumeran todas las combinaciones de uno, dos y tres bits. Con un bit existen 0 y 1. Con dos bits: 00, 01, 10 y 11. Con tres bits: 000, 001, 010, 011, 100, 101, 110 y 111. Todas se presentan con el mismo código de colores y tipografía. La secuencia 2, 4, 8 motiva la relación general  $N = 2^n$ .

# Bits y estados: dos caminos de ida y vuelta

Cada cubito representa un estado posible.



**$n$  bits  $\rightarrow 2^n$  estados**

**$N$  estados  $\rightarrow \lceil \log_2(N) \rceil$  bits**

5 / 19

Cada cubito representa una combinación, no un bit. Para 1–6 bits se muestran 2,4,8,16,32,64 estados. Ida: con  $n$  bits hay  $2^n$  estados. Vuelta: para  $N$  estados y código de longitud fija hacen falta  $\lceil \log_2 N \rceil$  bits. El gráfico muestra  $\log_2 N$ ; si no es entero, se redondea hacia arriba. Ejemplo: 64 estados  $\rightarrow$  6 bits; 10 estados  $\rightarrow$  4 bits. El caso de tres bits se puede enumerar oralmente: 000–111.

## ¿Cuántos bits necesitamos para diez símbolos?

Queremos asignar un código binario distinto, de igual longitud, a cada dígito del 0 al 9.

¿Cuál es el mínimo número de bits por código?

0 1 2 3 4 5 6 7 8 9

¿Bastan 3 bits?

¿Bastan 4 bits?

Ejercicio de un minuto. Se pide un código de longitud fija para cada dígito. No se trata de codificar un número arbitrariamente largo ni su representación textual.

## Buscamos la primera potencia de dos suficiente

Necesitamos al menos diez combinaciones distintas.

3 bits

$$2^3 = 8 \rightarrow \text{no bastan}$$

4 bits

$$2^4 = 16 \rightarrow \text{bastan}$$

Usamos 10 códigos; quedan 6 sin asignar.

**Para 10 símbolos hacen falta, como mínimo, 4 bits.**

FÓRMULAS APLICADAS

$$N_{\max} = 2^n \quad n_{\min} = \lceil \log_2 N \rceil$$

7 / 19

Primero razonar con potencias. La desigualdad general es  $2^n \geq N$ ;  $n$  entero mínimo =  $\lceil \log_2 N \rceil$  para  $N$  símbolos y código fijo. No hace falta desarrollar la función logarítmica aquí: queda en ampliación.

## ¿Cómo representamos «HAPPY»?

Usamos el siguiente código binario de longitud fija para representar las letras H, A, P e Y.

**H = 00**

**A = 01**

**P = 10**

**Y = 11**

Codifica el mensaje «HAPPY» e indica cuántos bits ocupa.  
¿Cómo recuperarías el mensaje a partir de su código?

El enunciado proporciona un código de dos bits por símbolo: H = 00, A = 01, P = 10 e Y = 11. Se pide codificar HAPPY, contar la longitud del mensaje y explicar cómo se recuperaría el texto. El resultado no se muestra hasta la diapositiva siguiente. Para decodificar se agrupan los bits de dos en dos y se aplica el mismo código.

## Codificamos y recuperamos el mensaje

Aplicamos el código H = 00, A = 01, P = 10 e Y = 11.

|    |    |    |    |    |
|----|----|----|----|----|
| H  | A  | P  | P  | Y  |
| 00 | 01 | 10 | 10 | 11 |

Para decodificar, necesitamos conocer el mismo código.

**HAPPY → 00 01 10 10 11 → 10 bits**

Mostrar la secuencia y leerla por parejas. 5 símbolos x 2 bits = 10 bits, solo para el mensaje, sin contar el coste de comunicar el código y la longitud. No es ASCII ni UTF-8. Decodificar 11 10 01 00 produce YPAH.

## El byte agrupa ocho bits

Una unidad habitual para medir y organizar datos.

0

1

0

0

0

1

1

1

**1 byte (B) = 8 bits (b)**

**8 bits  $\rightarrow 2^8 = 256$  combinaciones posibles**

Evitar confundir ocho bits con ocho combinaciones. Un byte contiene ocho bits y admite 256 patrones. No decir que un byte equivale siempre a un carácter.

## ASCII: un código estandar para los caracteres

Ejemplo: G24 B, guardando cada código ASCII en un byte.

| G        | 2        | 4        | espacio  | B        |
|----------|----------|----------|----------|----------|
| 01000111 | 00110010 | 00110100 | 00100000 | 01000010 |

**ASCII usa 7 bits; aquí cada código se guarda en 1 byte.**

ASCII representa 128 valores con 7 bits. Se muestran los códigos en bytes, con un cero inicial: G=01000111,2=00110010,4=00110100,espacio=00100000,B=01000010. El espacio cuenta como carácter. El ejemplo guardado así ocupa 5 bytes. No introducir UTF-8 en esta lección.

## Unidades: prefijo × bit o byte

Prefijos decimales del SI y prefijos binarios normalizados por la IEC.

| DECIMAL (SI) | FACTOR    | BINARIO (IEC) | FACTOR   |
|--------------|-----------|---------------|----------|
| kilo (k)     | $10^3$    | kibi (Ki)     | $2^{10}$ |
| mega (M)     | $10^6$    | mebi (Mi)     | $2^{20}$ |
| giga (G)     | $10^9$    | gibi (Gi)     | $2^{30}$ |
| tera (T)     | $10^{12}$ | tebi (Ti)     | $2^{40}$ |
| peta (P)     | $10^{15}$ | pebi (Pi)     | $2^{50}$ |
| exa (E)      | $10^{18}$ | exbi (Ei)     | $2^{60}$ |

**b = bit    B = byte    1 B = 8 b**

1 kb = 1 000 b    ·    1 KiB = 1 024 B    ·    1 MB =  $10^6$  B

12 / 19

Los prefijos decimales pertenecen al Sistema Internacional de Unidades (SI): kilo, mega y giga corresponden a potencias de diez. Los prefijos binarios normalizados por la Comisión Electrotécnica Internacional (IEC), como kibi, mebi y gibi, corresponden a potencias de dos. Debe distinguirse entre el prefijo y la unidad: b significa bit y B significa byte. Un byte contiene ocho bits. Por ejemplo, 1 MB =  $10^6$  bytes, mientras que 1 MiB =  $2^{20}$  bytes. Fuentes oficiales: BIPM, <https://www.bipm.org/en/measurement-units/si-prefixes> ; NIST, <https://physics.nist.gov/cuu/Units/binary.html> .

## «Tengo fibra de 100 megas»

Tienes contratada una conexión de 100 Mb/s. Al descargar un archivo, el programa indica 12,5 MB/s. ¿Recibes menos de lo contratado?

TU CONEXIÓN

**100 Mb/s**

TU DESCARGA

**12,5 MB/s**



**¿Te está llegando menos de lo contratado?**

Motivación: 100Mb/s contratados y 12,5MB/s en el gestor. Preguntar si son distintas antes de convertir. Son tasas equivalentes. Se simplifica suponiendo tasa útil constante; no analizar sobrecargas aquí.

## La misma tasa, expresada en otra unidad

1 byte = 8 bits

$$100 \text{ Mb/s} \div 8 = 12,5 \text{ MB/s}$$

La conexión y el programa están diciendo lo mismo.

### FÓRMULAS APLICADAS

$$R_{\text{B/s}} = \frac{R_{\text{bit/s}}}{8}$$

Se resuelve únicamente la comparación entre 100 Mb/s y 12,5 MB/s. Puesto que 1 byte = 8 bits, dividir la tasa expresada en bits/s entre 8 proporciona la tasa en bytes/s. Ambas cantidades representan la misma velocidad de transferencia. La pregunta sobre el tiempo de descarga se presenta por separado en la siguiente diapositiva.

## ¿Cuánto tarda en descargarse 1 GB?

Quieres descargar un archivo de 1 GB mediante una conexión de 100 Mb/s. Suponiendo una tasa constante y sin sobrecarga, ¿cuánto tiempo tardará la descarga?

**ARCHIVO**  
**1 GB**

**CONEXIÓN**  
**100 Mb/s**

Usamos unidades decimales: 1 GB = 1 000 MB.

Este es el segundo subproblema del ejercicio de la conexión de 100 megas. La comparación entre Mb/s y MB/s ya se ha resuelto. Ahora se plantea una pregunta distinta: cuánto tarda en descargarse un archivo de 1 GB a una tasa constante de 100 Mb/s. Se emplean prefijos decimales y se ignoran esperas, variaciones de tasa y sobrecargas. No se proporciona aún la resolución, que comienza convirtiendo el tamaño del archivo a megabits.

## Primer paso: usar unidades compatibles

Convertimos el tamaño del archivo a megabits.

$$1 \text{ GB} = 1\,000 \text{ MB}$$

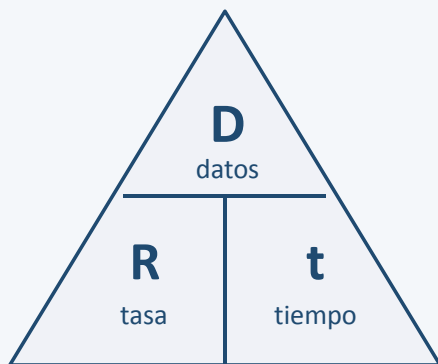
$$1\,000 \text{ MB} \times 8 = 8\,000 \text{ Mb}$$

**Ya tenemos Mb en el archivo y Mb/s en la conexión.**

1 GB decimal =  $10^9$  bytes =  $8 \cdot 10^9$  bits = 8000 Mb. Pedir al alumnado que justifique cada conversión; no dividir directamente 1 entre 100.

## Cantidad, tasa y tiempo

Como espacio = velocidad × tiempo, aquí: datos = tasa × tiempo.



FÓRMULAS APLICADAS

$$D = R t$$

$$R = \frac{D}{t}$$

$$t = \frac{D}{R}$$

$$t = 8\,000 \text{ Mb} / (100 \text{ Mb/s}) = 80 \text{ s}$$

Triángulo para despejar  $D=R \times t$ . No sustituye convertir unidades previamente. Tapar  $D:R \times t$ ; tapar  $R:D/t$ ; tapar  $t:D/R$ .  $D$ :cantidad de datos;  $R$ :tasa;  $t$ :tiempo.  $1\text{GB}=8000\text{Mb}$ ,  $80\text{s}=1\text{min}20\text{s}$  a tasa constante de  $100\text{Mb/s}$ .

## Comprobamos si lo hemos entendido

Tres preguntas rápidas para cerrar la lección.

- 1 ¿Cuántas combinaciones permiten 4 bits?
- 2 ¿Bastan 5 bits para distinguir 26 letras?
- 3 Para el mismo archivo, si duplicamos la tasa, ¿qué ocurre con el tiempo de descarga?

Respuestas: 1)16 combinaciones; 2)32 códigos bastan: cinco bits por símbolo, longitud fija; 3)el tiempo se reduce a la mitad si tamaño fijo y misma hipótesis de tasa constante. Pedir explicación oral; las respuestas quedan en notas para no saturar la diapositiva.

# Abramos el computador

¿Qué partes hacen posible almacenar, procesar y comunicar?



**¡Vamos a mirar  
dentro!**

**¿Dónde se guardan  
los datos?**

**¿Quién ejecuta  
las instrucciones?**

Transición a L1.2. Pedir hipótesis sobre memoria, procesador y entrada/salida, sin resolver todas las preguntas aquí. La ilustración sirve para despertar curiosidad; no es un plano técnico.

# L1.2

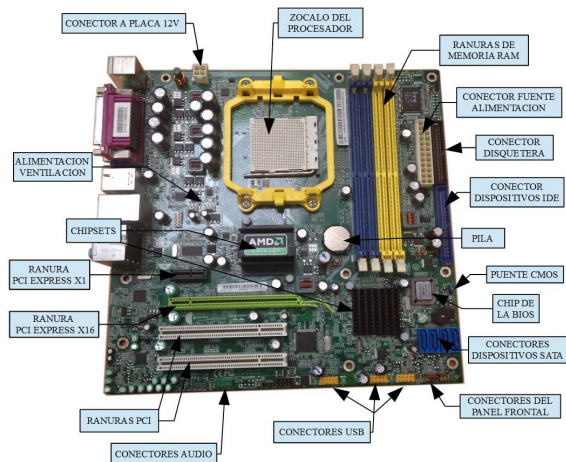
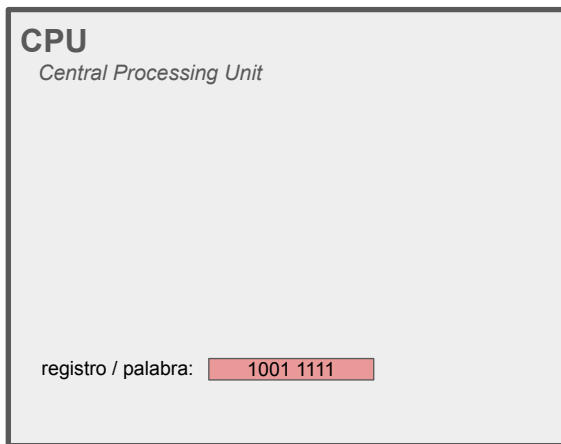
---

02

## Unidades funcionales

En esta lección se estudian las unidades que permiten a un computador almacenar información, ejecutar instrucciones e intercambiar datos. Partiremos de un procesador sencillo de 8 bits e iremos incorporando memoria, unidad de control, unidad aritmético-lógica y buses. Después seguiremos una operación paso a paso y relacionaremos el modelo con el computador completo. El modelo es didáctico: no pretende representar todos los detalles de un procesador real.

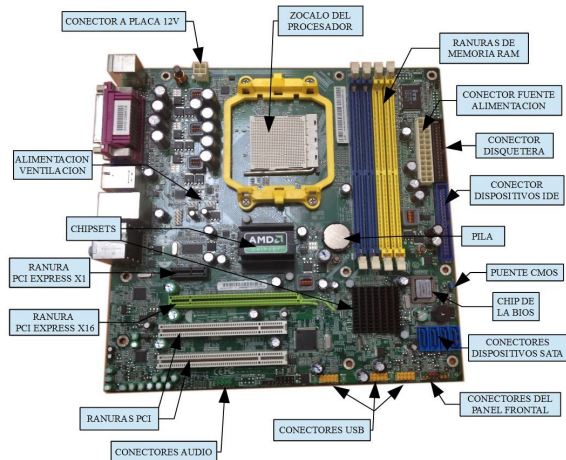
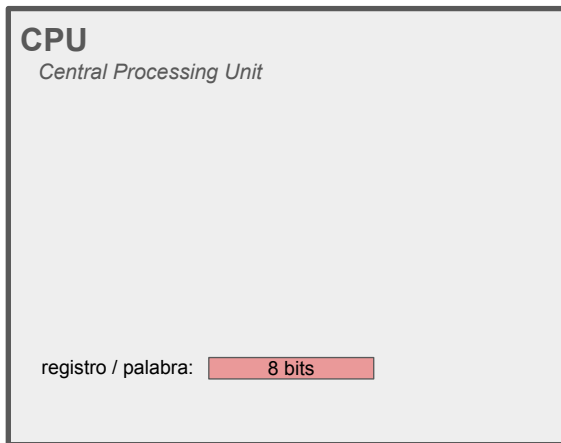
## La CPU trabaja con grupos de bits



1 / 18

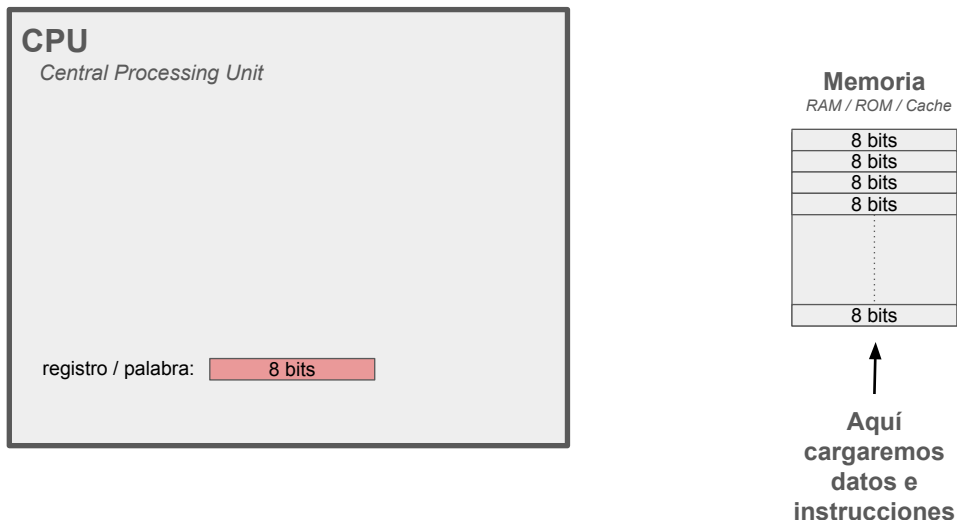
La CPU o unidad central de procesamiento ejecuta las instrucciones de los programas. En el dibujo aparece un registro, un pequeño almacenamiento interno que mantiene temporalmente un grupo de bits. El patrón 1001 1111 ocupa ocho posiciones binarias. Por sí solo no indica si representa un número, un carácter u otra información: su interpretación depende del código y de la operación. En este modelo utilizaremos grupos de ocho bits para poder seguir visualmente los movimientos de datos. Un procesador real dispone de varios registros y de más elementos de los que muestra esta primera figura.

## Ocho bits forman la palabra de nuestro modelo



La figura sustituye el contenido concreto del registro por su anchura: 8 bits. Así distinguimos dos ideas: el valor almacenado puede cambiar, mientras que la capacidad del registro permanece fija. La palabra de máquina es un grupo de bits que el procesador maneja como unidad natural en determinadas operaciones; aquí adoptamos una palabra de ocho bits. Es una simplificación del ejemplo, no una afirmación sobre todos los computadores. En otros procesadores son habituales palabras y registros de 32 o 64 bits, y pueden coexistir registros de distintas anchuras.

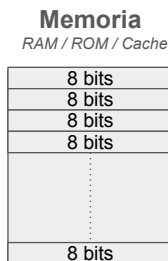
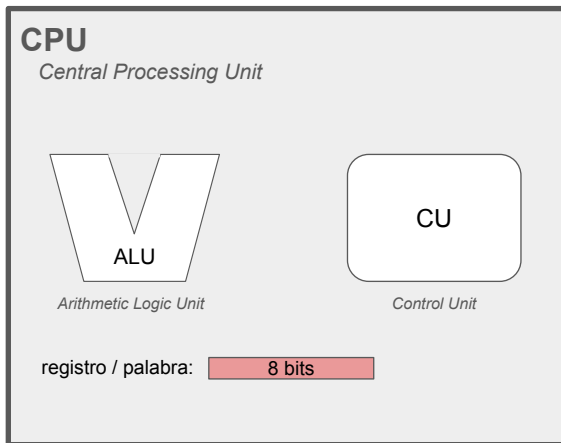
## La memoria guarda datos e instrucciones



3 / 18

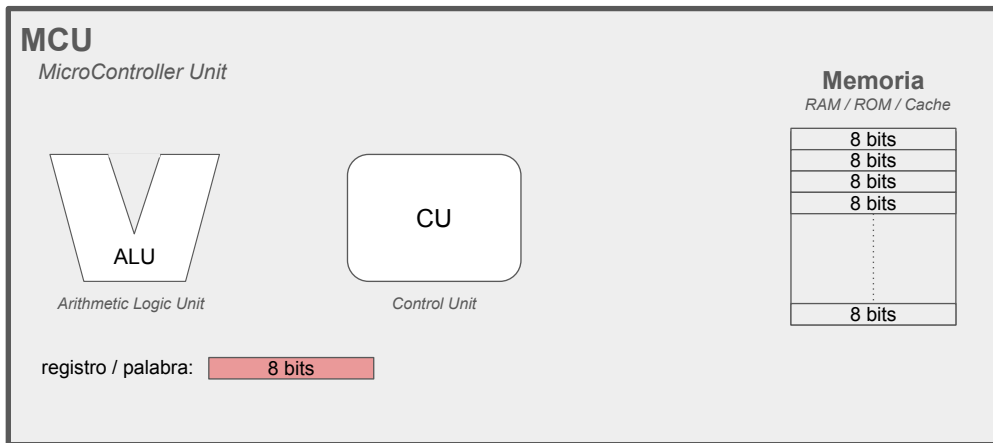
Incorporamos la memoria, donde se almacenan los datos y las instrucciones que necesita el programa. Cada fila del dibujo representa una posición de ocho bits en este modelo. Para acceder a una posición hay que identificarla mediante una dirección. La figura reúne las etiquetas RAM, ROM y caché para recordar distintos tipos de memoria, pero no son intercambiables: la RAM suele almacenar el programa y los datos en uso; la ROM o memoria no volátil puede contener firmware; la caché conserva copias de información para reducir tiempos de acceso. La secuencia posterior utiliza una memoria abstracta de lectura y escritura.

## La CPU calcula y coordina



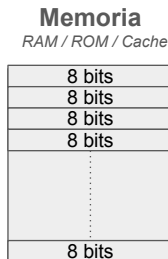
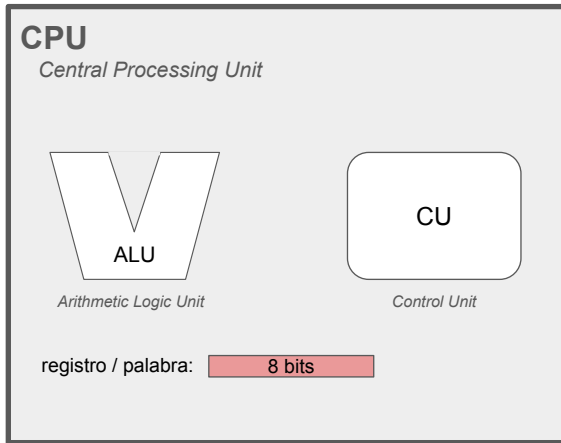
Dentro del procesador distinguimos la ALU y la unidad de control. La ALU, unidad aritmético-lógica, realiza operaciones como sumas, comparaciones y operaciones lógicas sobre bits. La unidad de control interpreta las instrucciones y coordina qué datos se leen, qué operación se realiza y dónde se guarda el resultado. Los registros mantienen valores temporales cerca de estas unidades. El dibujo separa responsabilidades: calcular no es lo mismo que decidir y coordinar los pasos. La CPU completa incluye estos elementos y otras estructuras que se omiten para facilitar la explicación.

## Un microcontrolador integra varias unidades



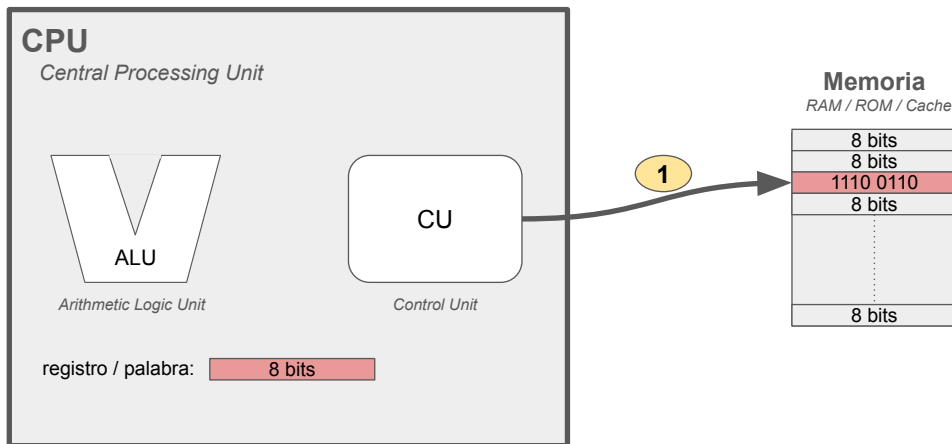
La figura muestra un microcontrolador, MCU: un circuito integrado que reúne un núcleo de procesamiento, memoria y, habitualmente, periféricos de entrada/salida y temporizadores. La idea central es la integración de funciones en un mismo chip. Resulta apropiado para controlar dispositivos y sistemas empujados. El esquema conserva los elementos ya conocidos, pero los encierra en una unidad mayor. No debe interpretarse como que cualquier CPU acompañada de memoria constituye por sí sola un microcontrolador, ni como que todos los microcontroladores tengan caché. Los periféricos habituales no se dibujan en esta vista simplificada.

## Seguimos una operación paso a paso



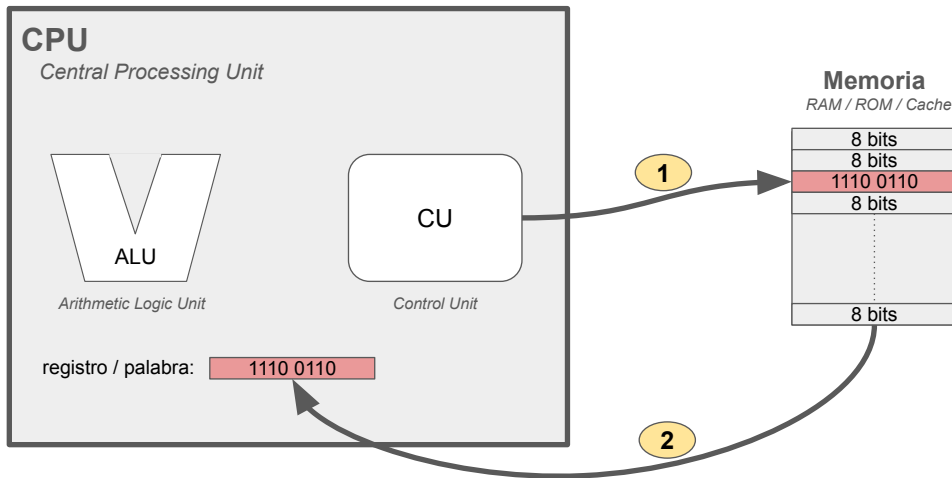
Volvemos al esquema con CPU y memoria separadas para seguir el recorrido de un dato. La operación que se ilustrará consiste en leer el patrón 1110 0110, invertir sus bits y guardar 0001 1001 en memoria. La secuencia permite distinguir selección de una posición, traslado de datos, procesamiento y escritura. Se trata de una operación didáctica sobre un dato; no es una descripción completa del ciclo de búsqueda y decodificación de instrucciones. Las instrucciones que ordenan estos movimientos y ciertos registros de control quedan implícitos en el dibujo.

## 1 • Seleccionar el dato en memoria



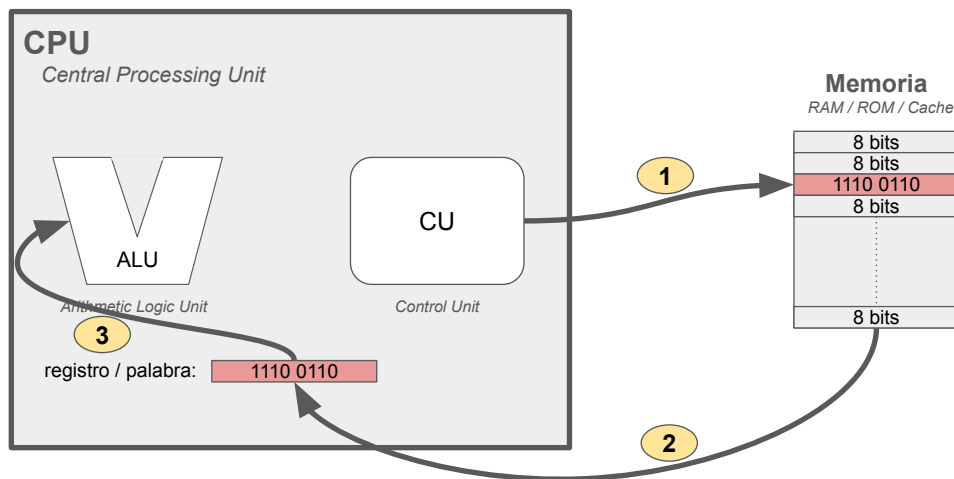
La unidad de control inicia la lectura seleccionando la posición de memoria que contiene 1110 0110. La flecha 1 representa la solicitud de acceso a esa posición. Conceptualmente, la dirección indica dónde está el dato y la señal de lectura indica qué se quiere hacer. El contenido todavía permanece en memoria: pedir una lectura no equivale a haber recibido el valor. En las diapositivas posteriores distinguiremos el papel del bus de direcciones y del bus de datos, que aquí se representan de forma simplificada.

## 2 · Llevar el dato a un registro



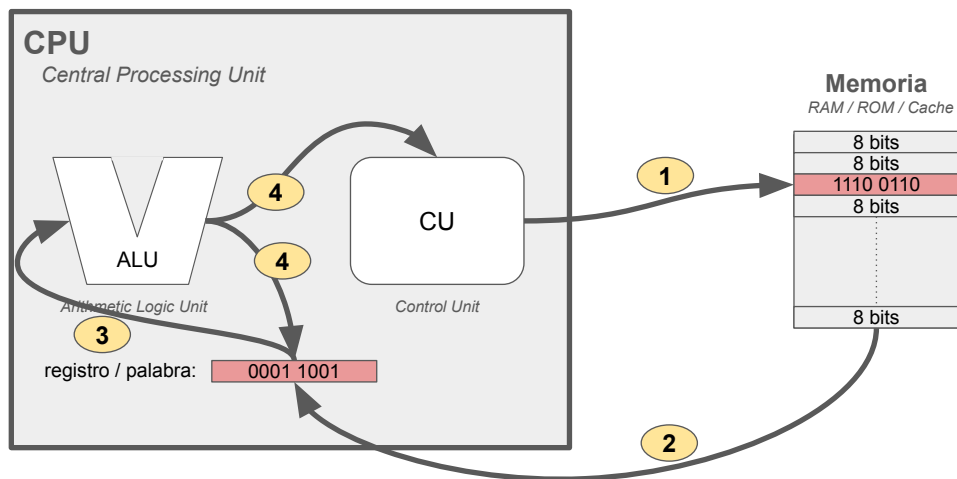
La memoria devuelve el contenido de la posición seleccionada. La flecha 2 muestra el traslado de 1110 0110 hacia un registro del procesador. El registro actúa como almacenamiento temporal para que el dato pueda participar en la operación. Una lectura normal copia el contenido: no borra ni vacía la posición de memoria de origen. Al terminar este paso, el mismo patrón puede estar presente en memoria y en el registro. La anchura de ocho bits del ejemplo permite ver el dato completo sin ocultar el significado de cada transferencia.

### 3 • Enviar el operando a la ALU



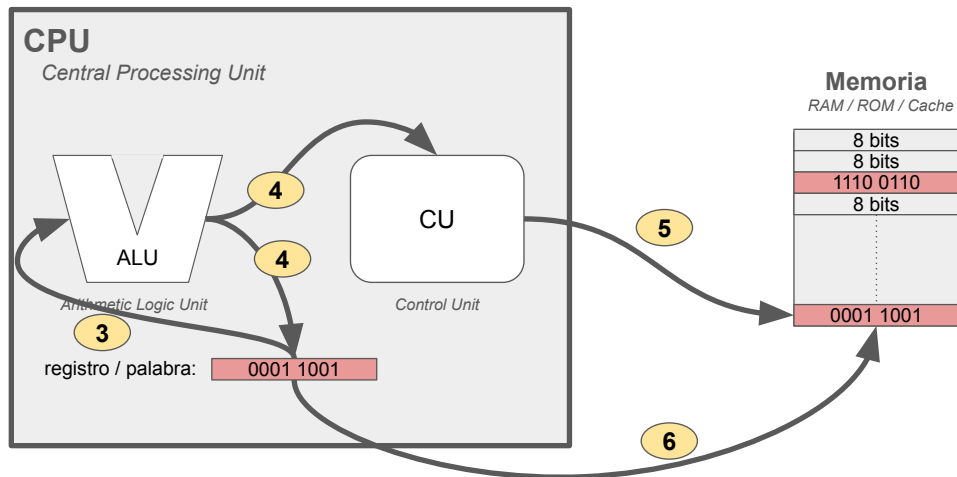
El contenido del registro se proporciona a la ALU mediante el recorrido marcado con 3. Ese contenido es el operando sobre el que se actuará. La unidad de control determina la operación que debe realizarse; la ALU no decide por sí misma qué instrucción ejecutar. En este ejemplo basta un operando porque se invertirá cada bit. Otras operaciones, como la suma, necesitan dos operandos. El dibujo muestra un único registro para mantener la secuencia sencilla, aunque un procesador real dispone de más registros y caminos internos.

## 4 • Operar y conservar el resultado



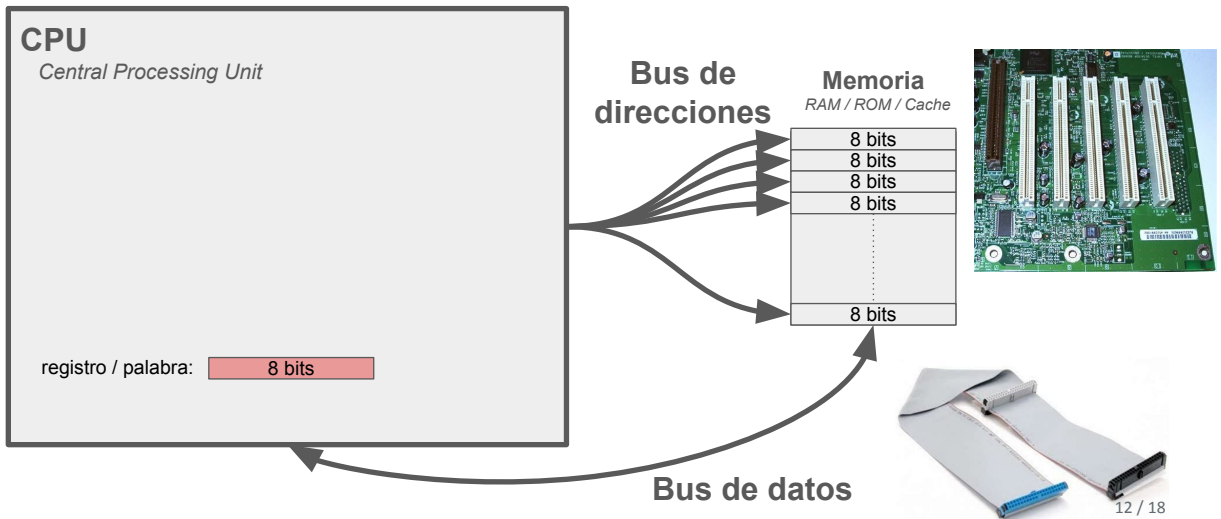
La ALU aplica una negación bit a bit: cada 1 pasa a 0 y cada 0 pasa a 1. Por eso 1110 0110 se convierte en 0001 1001. El resultado queda en el registro mostrado. Los recorridos marcados con 4 relacionan el cálculo, el almacenamiento temporal del resultado y la coordinación con la unidad de control. No representan necesariamente operaciones simultáneas ni un número concreto de ciclos de reloj. Invertir todos los bits tampoco equivale, en general, a obtener el negativo aritmético de un número: aquí estamos describiendo una operación lógica.

## 5–6 · Guardar el resultado en memoria



La unidad de control selecciona la posición de destino e inicia la escritura, representada por el paso 5. El paso 6 lleva el resultado 0001 1001 desde el registro a esa posición de memoria. Así se completa el recorrido leer–procesar–guardar. En la figura se usa una posición de destino distinta de la de origen, cuyo valor 1110 0110 se mantiene. La escritura sustituye el contenido anterior de la posición de destino. Como en la lectura, es útil separar la dirección y las señales que coordinan el acceso del dato que se transfiere.

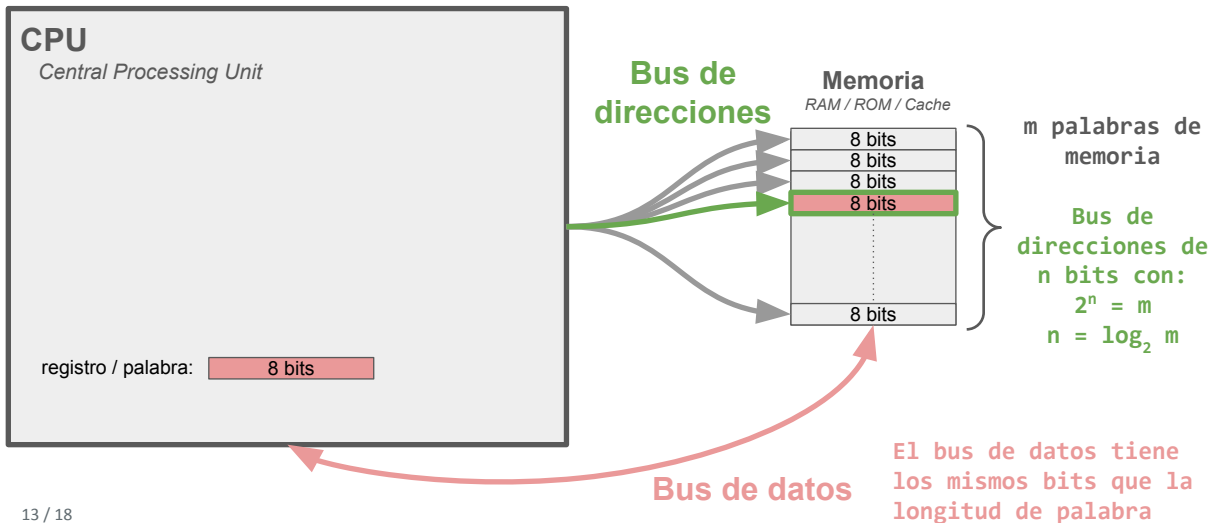
## Dos buses, dos funciones



12 / 18

El bus de direcciones identifica la posición de memoria a la que se desea acceder; el bus de datos transporta el contenido leído o escrito. La pregunta que responde el primero es «¿dónde?», mientras que el segundo responde «¿qué valor?». En el modelo CPU–memoria, el procesador proporciona la dirección. Los datos pueden ir de memoria a CPU en una lectura o de CPU a memoria en una escritura. Las señales de control, como lectura y escritura, también son necesarias aunque no se desarrollan en esta figura. Las flechas de direcciones ilustran la selección de posiciones, no un cable independiente para cada fila.

## Direcciones posibles y datos por transferencia



13 / 18

Con un bus de direcciones de  $n$  bits se pueden distinguir como máximo  $2^n$  direcciones. Si se utilizan todas y hay  $m$  posiciones,  $m=2^n$  y  $n=\log_2(m)$ . Si  $m$  no es una potencia de dos, el mínimo entero es  $n=\lceil \log_2(m) \rceil$ . La capacidad total depende además del tamaño de cada posición: número de posiciones por bits o bytes por posición. En esta figura cada posición contiene ocho bits. El texto del dibujo iguala la anchura del bus de datos a la palabra del procesador porque así se ha definido este modelo; esa igualdad no es una regla universal. Conviene distinguir anchura de registros, anchura del bus y unidad direccionable.

## El computador: cinco funciones conectadas

La CPU procesa; las memorias conservan; la entrada y salida comunican.



Esta vista reúne las funciones del computador. La entrada incorpora información del exterior y la salida comunica resultados. La CPU ejecuta las instrucciones del programa. La memoria principal mantiene los datos e instrucciones que están en uso, mientras que el almacenamiento conserva archivos y programas de forma persistente. Al abrir un programa, parte de su contenido se carga desde el almacenamiento a la memoria principal para poder ejecutarse. Las conexiones representan relaciones funcionales; no son un plano de cableado ni pretenden mostrar todos los controladores o buses reales. El objetivo es reconocer el papel de cada unidad sin confundir memoria principal y almacenamiento.

## Dentro de la CPU: guardar, calcular y coordinar

Registros, ALU y control colaboran siguiendo el ritmo del reloj.



Los registros almacenan operandos, resultados y otros valores internos durante la ejecución. La ALU efectúa operaciones aritméticas y lógicas: por ejemplo, sumar dos operandos o invertir los bits de uno. La unidad de control interpreta las instrucciones y coordina las transferencias y operaciones; su acción alcanza a toda la CPU, aunque aquí solo se muestra una conexión representativa para no recargar la figura. El reloj proporciona una referencia temporal para coordinar cambios de estado. Su periodo  $T$  es el tiempo de un ciclo y su frecuencia es  $f=1/T$ . Una instrucción puede necesitar varios ciclos; también puede haber varias instrucciones solapadas. Por eso frecuencia de reloj e instrucciones por segundo no son la misma magnitud.

## Dentro de la CPU: guardar, calcular y coordinar

Regist

Gua  
t

ROL

eta y  
ina

RELOJ

re a una

16 / 18

Los registros almacenan operandos, resultados y otros valores internos durante la ejecución. La ALU efectúa operaciones aritméticas y lógicas: por ejemplo, sumar dos operandos o invertir los bits de uno. La unidad de control interpreta las instrucciones y coordina las transferencias y operaciones; su acción alcanza a toda la CPU, aunque aquí solo se muestra una conexión representativa para no recargar la figura. El reloj proporciona una referencia temporal para coordinar cambios de estado. Su periodo  $T$  es el tiempo de un ciclo y su frecuencia es  $f=1/T$ . Una instrucción puede necesitar varios ciclos; también puede haber varias instrucciones solapadas. Por eso frecuencia de reloj e instrucciones por segundo no son la misma magnitud.

## Frecuencia de reloj y tiempo de ejecución

Más GHz no garantizan menos tiempo para cualquier programa.

### FRECUENCIA $f$

Ciclos por segundo  
 $2 \text{ GHz} = 2 \times 10^9 \text{ ciclos/s}$

### CPI MEDIO

Ciclos por instrucción  
 Valor medio del programa

$$\text{Tiempo de CPU} = NI \times CPI / f$$

NI: instrucciones ejecutadas. Para la misma tarea, menos tiempo es mejor.

17 / 18

La frecuencia  $f$  cuenta ciclos por segundo: 1 Hz es un ciclo por segundo y 1 GHz son mil millones de ciclos por segundo. El CPI medio es el número de ciclos consumidos por instrucción, promediado sobre las instrucciones ejecutadas por un programa. Si NI es ese número de instrucciones, el total de ciclos es  $NI \times CPI$  y el tiempo de CPU resulta  $NI \times CPI / f$ . Por ejemplo, con  $NI = 2 \times 10^9$ ,  $CPI = 2$  y  $f = 2 \times 10^9 \text{ ciclos/s}$ , el tiempo es 2 segundos. El programa, el compilador, la arquitectura, los accesos a memoria y las dependencias pueden cambiar NI o CPI. En procesadores que solapan varias instrucciones, el CPI medio puede ser inferior a uno. El tiempo de CPU no incluye necesariamente toda la espera de disco, red o interacción humana.

## Rendimiento: instrucciones y operaciones por segundo

### MIPS

Millones de instrucciones por segundo

$$\text{MIPS} = NI / (t \times 10^6)$$

También:  $f / (\text{CPI} \times 10^6)$

### MFLOPS

Millones de operaciones en coma flotante por segundo

$$\text{MFLOPS} = NF / (t \times 10^6)$$

NF: operaciones en coma flotante

**Para comparar equipos: misma tarea y menor tiempo de ejecución.**

18 / 18

MIPS significa millones de instrucciones máquina ejecutadas por segundo. Se calcula como  $NI/(t \times 10^6)$ , donde  $t$  es el tiempo de CPU en segundos. Si se conocen frecuencia en Hz y CPI medio, también puede calcularse como  $f/(\text{CPI} \times 10^6)$ . Por ejemplo, 2 GHz y  $\text{CPI}=2$  dan 1000 MIPS bajo esas condiciones. Una instrucción no realiza necesariamente el mismo trabajo en distintas arquitecturas, por lo que MIPS no sirve como clasificación universal de rapidez. FLOPS cuenta operaciones en coma flotante por segundo; MFLOPS expresa millones de esas operaciones. NF es el número de operaciones de coma flotante realizadas. No son necesariamente operaciones de 64 bits: deben especificarse la precisión y el tipo de medida. Una instrucción vectorial puede realizar varias operaciones. Para una comparación útil deben mantenerse la tarea, las condiciones y, cuando corresponda, la precisión; el tiempo de ejecución de esa tarea es la referencia principal.

# L1.3

---

03

## Tipos de computadores

Los computadores comparten las unidades funcionales vistas en L1.2, pero se adaptan a necesidades diferentes. En esta lección distinguimos tres criterios: generalidad de uso, escala de trabajo y paralelismo. No son categorías excluyentes ni una clasificación única. El objetivo es reconocer las diferencias principales con ejemplos cotidianos, sin estudiar arquitecturas concretas ni memorizar precios o cifras de potencia.

## Distintos computadores, distintas necesidades

### SEGÚN SU USO

#### Uso general

Muchas aplicaciones:  
PC y móvil.



#### Uso especializado

Una función:  
controlar un dispositivo.



### SEGÚN SU ESCALA Y EL TRABAJO QUE REALIZAN

#### Equipo personal



Aplicaciones de  
una persona

#### Servidor / mainframe



Servicios para muchos  
usuarios

#### Supercomputador



Grandes cálculos  
y simulaciones

Son criterios complementarios: un mismo equipo encaja en varios.

1 / 2

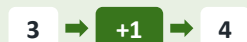
Por su uso, un computador puede ser de propósito general, como un PC o un teléfono inteligente que ejecuta muchas aplicaciones, o estar especializado en una función, como el controlador de un motor. Los computadores empotrados forman parte de otro dispositivo. Por su escala y carga de trabajo distinguimos equipos personales, servidores y grandes sistemas de cálculo. Un servidor proporciona servicios a otros equipos; un mainframe suele priorizar grandes volúmenes de transacciones y fiabilidad; un supercomputador se orienta a cálculos y simulaciones de gran tamaño. Estas categorías se solapan: no se definen mediante un precio fijo ni un número concreto de procesadores. Un móvil puede ser de propósito general y un servidor puede tener muchos núcleos.

## Paralelismo: repartir el trabajo

### SISD

#### Una secuencia

Una instrucción sobre un dato cada vez.



### SIMD

#### La misma operación

Una instrucción sobre varios datos.



### MIMD

#### Tareas diferentes

Varias instrucciones sobre distintos datos.



MIMD: varios núcleos en un equipo o varios computadores conectados.

Un computador actual puede combinar varias formas de paralelismo.

2 / 2

El paralelismo permite realizar varias partes del trabajo a la vez. SISD representa un único flujo de instrucciones sobre un único flujo de datos. SIMD aplica una misma instrucción a varios datos: por ejemplo, sumar una unidad a todos los elementos de un vector. MIMD permite que distintas unidades ejecuten instrucciones diferentes sobre datos diferentes. Puede organizarse con varios núcleos que comparten memoria o con computadores completos que intercambian mensajes. Estas formas pueden coexistir: un procesador multinúcleo puede ejecutar programas distintos en sus núcleos y utilizar instrucciones SIMD dentro de cada uno. Se muestran las tres formas del material original; no se pretende desarrollar aquí toda la taxonomía de Flynn ni equiparar de forma rígida un tipo de equipo con una sola categoría.

# L1.4

---

04

## Software de un computador

El hardware proporciona los componentes físicos y el software indica qué operaciones deben realizar. Esta lección introduce instrucciones y programas, y distingue dos etapas que conviene no confundir: la traducción de un programa y su ejecución. Se utiliza el cálculo del importe de una compra como hilo conductor.

# El software indica qué hacer al hardware

Software: los programas que utiliza un computador.

## INSTRUCCIÓN

Una orden para realizar una operación sobre los datos.

## PROGRAMA

Instrucciones organizadas para realizar una tarea.

El software comprende los programas que utiliza un computador. Una instrucción expresa una orden para realizar una operación; un programa organiza instrucciones para realizar una tarea. Puede incluir secuencias, decisiones y repeticiones. Una instrucción de alto nivel no tiene por qué corresponder a una sola instrucción máquina. El ejemplo de calcular un importe se reserva para la siguiente diapositiva, donde permite distinguir traducción y ejecución.

# Del programa al resultado

## 1 · TRADUCIR

### Programa fuente

importe =  
unidades × precio



### Compilador

Traduce las  
instrucciones



### Programa ejecutable

0101 1100  
1010 0011

Código máquina\*

Se carga para ejecutarlo



## 2 · EJECUTAR

### Datos

3 unidades · 2 €



### Computador

Ejecuta el programa



### Resultado

6 €

\* Bits ilustrativos. Se muestra el caso de un programa compilado.

2 / 2

El esquema distingue traducción y ejecución en el caso sencillo de un programa compilado. En la primera etapa, un compilador, que también es un programa ejecutado por el computador, traduce el programa fuente a código máquina; pueden intervenir otras herramientas como el enlazador. El resultado es un programa ejecutable. En la segunda etapa, el ejecutable se carga y se ejecuta con los datos de entrada: con 3 unidades a 2 euros por unidad se obtiene un importe de 6 euros. Los bits dibujados son ilustrativos y no codifican realmente la expresión mostrada. No todos los lenguajes siguen exactamente este proceso: existen intérpretes, máquinas virtuales y compilación durante la ejecución. El objetivo es comprender que traducir genera una representación ejecutable y ejecutar produce resultados a partir de los datos.

# Ejercicios

## Aplicamos los conceptos

Tiempo de ejecución · Memoria y buses · Rendimiento

Este bloque distingue los planteamientos, identificados en azul, de las resoluciones, identificadas en ámbar. Se recomienda intentar cada ejercicio antes de pasar a la siguiente diapositiva. Los cuatro problemas relacionan instrucciones, ciclos, frecuencia, capacidad y tasas de transferencia. Los diagramas del ejercicio 2 se conservan para razonar sobre las palabras de memoria y los buses.

## Un pequeño repaso

- 1 ¿En qué se diferencian MIPS y MFLOPS?
- 2 ¿Qué es una operación en coma flotante?
- 3 ¿Qué indica la longitud de palabra?

MIPS mide millones de instrucciones ejecutadas por segundo. MFLOPS mide millones de operaciones en coma flotante por segundo. Una operación en coma flotante opera sobre números representados mediante significando y exponente; no equivale necesariamente a una instrucción máquina. La longitud de palabra indica cuántos bits forman una palabra del modelo considerado. No debe confundirse con la capacidad total de memoria ni con el número de direcciones. Los valores de MIPS dependen del programa y del repertorio de instrucciones.

## Ejercicio 1 · Tiempo de ejecución

Un procesador tarda, en promedio, 4 ciclos de reloj en ejecutar cada instrucción máquina y funciona a 1 GHz. ¿Cuánto tardará en ejecutar un programa de 10 000 instrucciones?

$$NI = 10\,000 \quad \cdot \quad CPI = 4 \quad \cdot \quad f = 1\text{ GHz}$$

- a) 0,25  $\mu$ s      b) 40  $\mu$ s
- c) 40 000 s      d)  $40 \times 10^{12}$  s

Se pide el tiempo de CPU de un programa que ejecuta 10 000 instrucciones con CPI medio 4 y frecuencia de 1 GHz. El CPI ya es un promedio del programa. Antes de calcular, distinguir instrucciones, ciclos e intervalos de tiempo. Las cuatro opciones se conservan del enunciado original. No se añade tiempo de espera de entrada/salida.

## Ejercicio 1 · De instrucciones a tiempo

### 1 · Contamos los ciclos necesarios

10 000 instrucciones  $\times$  4 ciclos/instrucción = 40 000 ciclos

### 2 · Dividimos por los ciclos que se ejecutan cada segundo

$$t = N_I \times \text{CPI} / f = 40\,000 / 10^9 \text{ s}$$

$$t = 40 \times 10^{-6} \text{ s} = 40 \mu\text{s} \rightarrow \text{opción b)}$$

FÓRMULAS APLICADAS

$$N_c = N_I \text{CPI}$$

$$t = \frac{N_c}{f} = \frac{N_I \text{CPI}}{f}$$

El número total de ciclos es  $N_I \times \text{CPI} = 10\,000 \times 4 = 40\,000$ . La frecuencia de 1 GHz significa  $10^9$  ciclos por segundo. Al dividir ciclos entre ciclos por segundo obtenemos segundos:  $40\,000/10^9 = 0,00004 \text{ s} = 40 \text{ microsegundos}$ . Comprobación alternativa: cada ciclo dura 1 ns y 40 000 ciclos duran 40 000 ns, es decir, 40  $\mu\text{s}$ . La respuesta correcta es b).

## Ejercicio 1 · De instrucciones a tiempo

### 1 · Contamos los ciclos necesarios

10 000 instrucciones  $\times$  4 ciclos/instrucción = 40 000

### 2 · Dividimos por los ciclos que se ejecutan cada

$$t = N_I \times CPI / f = 40\,000 / 10^9 \text{ s}$$

$$t = 40 \times 10^{-6} \text{ s} = 40 \mu\text{s} \rightarrow \text{opción b)}$$

FÓRMULAS APLICADAS

$$N_c = N_I \text{ CPI}$$

$t$

### Prefijos del sistema internacional

|              | Factor     | Nombre | Símbolo |
|--------------|------------|--------|---------|
| MÚLTIPLOS    | $10^{24}$  | yotta  | Y       |
|              | $10^{21}$  | zetta  | Z       |
|              | $10^{18}$  | exa    | E       |
|              | $10^{15}$  | peta   | P       |
|              | $10^{12}$  | tera   | T       |
|              | $10^9$     | giga   | G       |
|              | $10^6$     | mega   | M       |
|              | $10^3$     | kilo   | k       |
|              | $10^2$     | hecto  | h       |
| SUBMÚLTIPLOS | $10^1$     | deca   | da      |
|              | $10^{-1}$  | deci   | d       |
|              | $10^{-2}$  | centi  | c       |
|              | $10^{-3}$  | mili   | m       |
|              | $10^{-6}$  | micro  | $\mu$   |
|              | $10^{-9}$  | nano   | n       |
|              | $10^{-12}$ | pico   | p       |
|              | $10^{-15}$ | femto  | f       |
|              | $10^{-18}$ | atto   | a       |
|              | $10^{-21}$ | zepto  | z       |
|              | $10^{-24}$ | yocto  | y       |

El número total de ciclos es  $N_I \times CPI = 10\,000 \times 4 = 40\,000$ . La frecuencia de 1 GHz significa  $10^9$  ciclos por segundo. Al dividir ciclos entre ciclos por segundo obtenemos segundos:  $40\,000/10^9 = 0,00004 \text{ s} = 40 \text{ microsegundos}$ . Comprobación alternativa: cada ciclo dura 1 ns y 40 000 ciclos duran 40 000 ns, es decir, 40  $\mu\text{s}$ . La respuesta correcta es b).

## Ejercicio 2 · Memoria y buses

Un computador tiene una capacidad máxima de memoria de 32 MiB, organizada en palabras de 32 bits. Cada dirección identifica una palabra y el bus de datos transfiere una palabra completa a la vez.

- a) ¿Cuántos bits puede almacenar en total?
- b) ¿Cuántas líneas tienen los buses de datos y de direcciones?
- c) Si el ancho de banda es de 200 MB/s, ¿qué tasa debe admitir cada línea del bus de datos?

32 MiB de capacidad · 32 bits/palabra · 200 MB/s de tasa total  
1 MiB =  $2^{20}$  B; 1 MB/s =  $10^6$  B/s.

4 / 15

Se explicitan los supuestos usados en la resolución original: las direcciones identifican palabras de 32 bits y el bus de datos transporta una palabra completa a la vez. Así se puede deducir un bus de direcciones de 23 bits. Si las direcciones identificaran bytes, serían necesarios 25 bits. Se corrige la notación de la capacidad a 32 MiB porque el cálculo original usa potencias de dos. La tasa se mantiene como 200 MB/s decimales. En el apartado c se supone transferencia en paralelo por las 32 líneas, con un bit por línea en cada transferencia y sin sobrecarga. Estas hipótesis son las del ejercicio, no propiedades universales de todos los computadores.

## Ejercicio 2 a) · Capacidad en bits

Primero expresamos la capacidad en bytes.

$$32 \text{ MiB} = 32 \times 2^{20} \text{ B} = 2^{25} \text{ B}$$

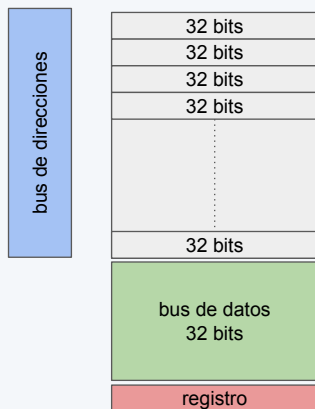
Cada byte contiene 8 bits.

$$2^{25} \text{ B} \times 8 \text{ bits/B} = 2^{28} \text{ bits}$$

268 435 456 bits = 256 Mibit

La capacidad binaria de 32 MiB equivale a  $32 \times 2^{20} = 2^{25}$  bytes. Al multiplicar por 8 bits por byte se obtienen  $2^{28} = 268\,435\,456$  bits. También puede expresarse como 256 Mibit. No son 256 Mbit decimales. La longitud de palabra no es necesaria para calcular la capacidad total en bits; se usará en el siguiente apartado para contar posiciones direccionables.

## Ejercicio 2 b) · Bus de datos

**Una palabra**

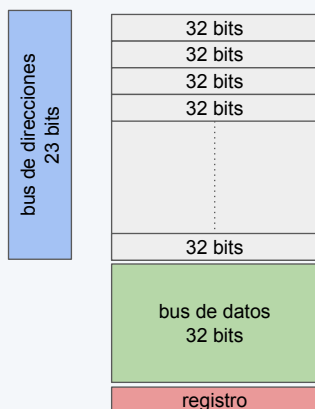
32 bits = 4 bytes

El bus transporta una palabra completa en cada transferencia.

**Bus de datos: 32 líneas**

El diagrama original representa palabras de memoria de 32 bits, un bus de datos y un registro. En el modelo del enunciado, una transferencia mueve una palabra completa en paralelo. Cada línea transporta un bit de esa palabra, de modo que son necesarias 32 líneas de datos. La capacidad total de 32 MiB no determina este ancho. Se conserva el diagrama original y se amplía de forma uniforme para facilitar su lectura.

## Ejercicio 2 b) · Bus de direcciones



**Contamos palabras, no bytes**

$$32 \text{ MiB} = 2^{25} \text{ bytes}$$

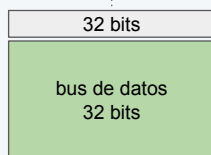
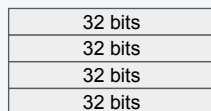
$$32 \text{ bits/palabra} = 4 \text{ bytes/palabra}$$

$$N = 2^{25} / 4 = 2^{23} \text{ palabras}$$

**Bus de direcciones: 23 bits**

Cada dirección selecciona una palabra de 4 bytes. La memoria contiene  $2^{25}$  bytes, por lo que hay  $2^{25}/4 = 2^{23}$  palabras. Para distinguir  $2^{23}$  posiciones hacen falta 23 bits de dirección. Se conserva el diagrama original con sus etiquetas de buses de 32 y 23 bits. La respuesta depende de que la memoria sea direccionable por palabras. Con direccionamiento por bytes, los mismos 32 MiB requerirían 25 bits de dirección.

## Ejercicio 2 c) · De bytes/s a bits/s



Tasa total: 200 MB/s

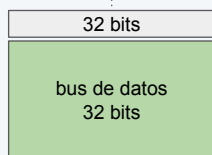
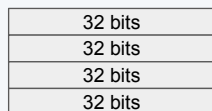
### Tasa de todo el bus

$$200 \text{ MB/s} \times 8 \text{ bits/B} \\ = 1\,600 \text{ Mbit/s}$$

Esta tasa se reparte entre las 32 líneas del bus de datos.

El diagrama original muestra palabras y un bus de datos de 32 bits. El ancho de banda dado es el total del bus, expresado en bytes por segundo. Convertimos primero a bits por segundo:  $200 \times 8 = 1\,600 \text{ Mbit/s}$ . En este modelo ideal, las 32 líneas trabajan en paralelo y contribuyen por igual a la transferencia. Por tanto, la tasa por línea se obtiene dividiendo la tasa total entre 32. La tasa decimal de 200 MB/s no debe confundirse con una capacidad ni con 200 MiB/s.

## Ejercicio 2 c) · Tasa por línea



Tasa total: 200 MB/s

**Repartimos la tasa total**

$$1\,600\text{ Mbit/s} \div 32\text{ líneas}$$

**50 Mbit/s por línea**

Comprobación:  $32 \times 50 = 1\,600\text{ Mbit/s}$

Cada línea debe admitir  $1\,600/32 = 50\text{ Mbit/s}$ , es decir, 50 millones de bits por segundo. Al multiplicar por 32 recuperamos la tasa total de  $1\,600\text{ Mbit/s} = 200\text{ MB/s}$ . El resultado expresa una tasa de bits, no una frecuencia de reloj determinada: su relación con el reloj depende del número de transferencias por ciclo y de la señalización. Se mantiene el diagrama creado por el profesor, con el bus de 32 bits, y se presenta el cálculo al lado.

## Ejercicio 3 · Prestaciones del procesador

Un programa de prueba se ejecuta en un procesador a una tasa de 100 MIPS y tarda 35 segundos. Cada instrucción consume, por término medio, 6 ciclos de reloj.

**100 MIPS   ·   35 s   ·   CPI medio = 6**

- a) Calcula la frecuencia de reloj.
- b) Calcula cuántas instrucciones se han ejecutado.

El procesador alcanza 100 millones de instrucciones máquina por segundo al ejecutar este programa, cuyo tiempo de CPU es 35 segundos y cuyo CPI medio es 6. Se pide obtener primero la frecuencia y después el número dinámico de instrucciones ejecutadas. Los valores se refieren al mismo programa y a la misma ejecución. Aunque el programa se haya escrito en un lenguaje de alto nivel, MIPS cuenta instrucciones máquina, no líneas de código fuente.

## Ejercicio 3 a) · Frecuencia de reloj

Partimos de la relación entre MIPS, frecuencia y CPI.

$$\text{MIPS} = \frac{f}{\text{CPI} \times 10^6}$$

$$f = \text{MIPS} \times \text{CPI} \times 10^6$$

Sustituimos:  $100 \times 6 \times 10^6$  ciclos/s

$$f = 600 \times 10^6 \text{ Hz} = 600 \text{ MHz}$$

La frecuencia se obtiene multiplicando las instrucciones por segundo por los ciclos por instrucción. Las unidades instrucción se cancelan y quedan ciclos por segundo:  $100 \times 10^6 \times 6 = 600 \times 10^6$  ciclos/s = 600 MHz. Equivale a despejar  $f$  en  $\text{MIPS} = f/(\text{CPI} \times 10^6)$ . La frecuencia calculada es 0,6 GHz. No deben identificarse MIPS y MHz sin tener en cuenta el CPI.

## Ejercicio 3 b) · Instrucciones ejecutadas

Relacionamos los MIPS con el tiempo de ejecución.

$$\text{MIPS} = \frac{f}{\text{CPI} \times 10^6} \Rightarrow \frac{f}{\text{CPI}} = \text{MIPS} \times 10^6$$

$$t = \frac{N_I \text{CPI}}{f} \Rightarrow N_I = t \frac{f}{\text{CPI}}$$

$$N_I = t \times \text{MIPS} \times 10^6$$

$$NI = 35 \times 100 \times 10^6 = 3,5 \times 10^9 \text{ instrucciones}$$

Partimos de  $\text{MIPS} = f/(\text{CPI} \times 10^6)$ . Por tanto,  $f/\text{CPI} = \text{MIPS} \times 10^6$ . La expresión del tiempo de CPU  $t = N_I \times \text{CPI}/f$  puede despejarse como  $N_I = t \times f/\text{CPI}$ . Sustituyendo la primera relación se deduce  $N_I = t \times \text{MIPS} \times 10^6$ . Para 35 segundos y 100 MIPS resultan  $3,5 \times 10^9$  instrucciones. La frecuencia  $f = 600 \text{ MHz}$  obtenida en el apartado a) permite verificarlo:  $N_I = 35 \times 600 \times 10^6/6 = 3,5 \times 10^9$ . Así se conecta el resultado con las fórmulas que ya se han presentado.

## Ejercicio 4 · Un programa con un bucle

Un programa contiene 200 instrucciones máquina; 150 forman un bucle que se ejecuta 50 veces. Las restantes se ejecutan una vez. La ejecución completa consume 34 000 ciclos y tarda 34  $\mu$ s.

**Ciclos totales: 34 000      ·      Tiempo: 34  $\mu$ s**

- a) Calcula el CPI medio.
- b) Calcula la frecuencia de reloj.

Ejercicio adaptado del problema P1.7 de Conceptos de Informática (Schaum), presente en el material original. Se supone que las 50 instrucciones fuera del bucle se ejecutan una vez y las 150 del bucle se ejecutan exactamente 50 veces cada una. Las instrucciones de control del bucle están incluidas en el recuento dado. No se deben usar las 200 instrucciones escritas como si fueran el número total de instrucciones ejecutadas. Se proporcionan 34 000 ciclos totales y un tiempo de CPU de 34 microsegundos.

## Ejercicio 4 a) · ¿Cuántas se ejecutan?

### FUERA DEL BUCLE

$$200 - 150 = 50$$

Se ejecutan una vez.

### DENTRO DEL BUCLE

$$150 \times 50 = 7\,500$$

Se repiten 50 veces.

$$\text{NI} = 50 + 7\,500 = 7\,550 \text{ instrucciones ejecutadas}$$

Repartimos los 34 000 ciclos entre esas instrucciones:

$$\text{CPI} = \frac{\text{ciclos totales}}{\text{instrucciones ejecutadas}} = \frac{34\,000}{7\,550} \approx 4,50$$

Las 200 instrucciones que contiene el programa no son el total de instrucciones que se ejecutan. Fuera del bucle hay  $200 - 150 = 50$  instrucciones, ejecutadas una vez cada una. Las 150 del bucle se ejecutan 50 veces, aportando 7 500 ejecuciones. El total es  $\text{NI} = 50 + 7\,500 = 7\,550$ . Como el programa consume 34 000 ciclos, dividir los ciclos totales entre las instrucciones ejecutadas da el CPI medio:  $34\,000 / 7\,550 = 4,5033\dots$ , aproximadamente 4,50 ciclos/instrucción. La frecuencia se obtiene en la siguiente diapositiva, usando directamente el total de ciclos y el tiempo para evitar el error de redondeo del CPI.

## Ejercicio 4 b) · Ciclos por segundo

El programa consume 34 000 ciclos en 34  $\mu$ s.

La frecuencia indica cuántos ciclos se realizan en un segundo.

**Primero: 34  $\mu$ s =  $34 \times 10^{-6}$  s**

$$f = \frac{\text{ciclos totales}}{\text{tiempo}} = \frac{N_c}{t}$$

$$f = \frac{34\,000}{34 \times 10^{-6} \text{ s}} = 10^9 \text{ Hz} = 1 \text{ GHz}$$

15 / 15

La frecuencia es el número de ciclos de reloj por segundo. Si la ejecución ocupa 34 000 ciclos y dura 34 microsegundos, se convierten los microsegundos a segundos y se divide:  $f = 34\,000 / (34 \times 10^{-6}) = 10^9 \text{ Hz} = 1 \text{ GHz}$ . Esta deducción usa directamente la definición de frecuencia. No es necesario emplear el CPI redondeado del apartado anterior. Comprobación: a 1 GHz cada ciclo dura 1 ns; 34 000 ciclos duran 34 000 ns = 34  $\mu$ s.

¡Gracias por vuestra atención!